

计算机设计与实践 流水线CPU及SoC设计

2026 · 夏

哈工大



HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HITSZ

目录



引言

流水线概述

流水线设计

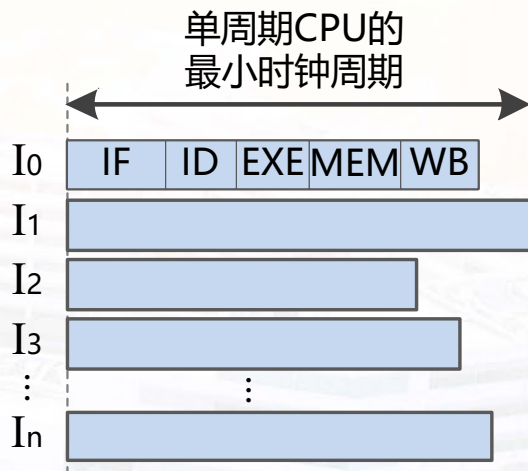
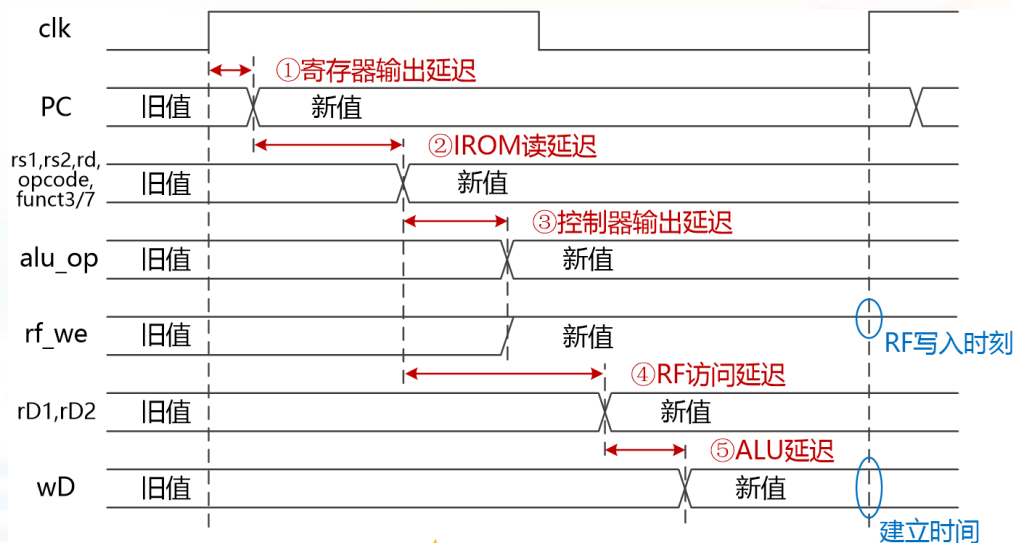
SoC架构及系统总线

I/O接口电路

引言

□ 单周期CPU性能上限是？

- 主要问题：时钟周期长（频率低）、指令串行执行

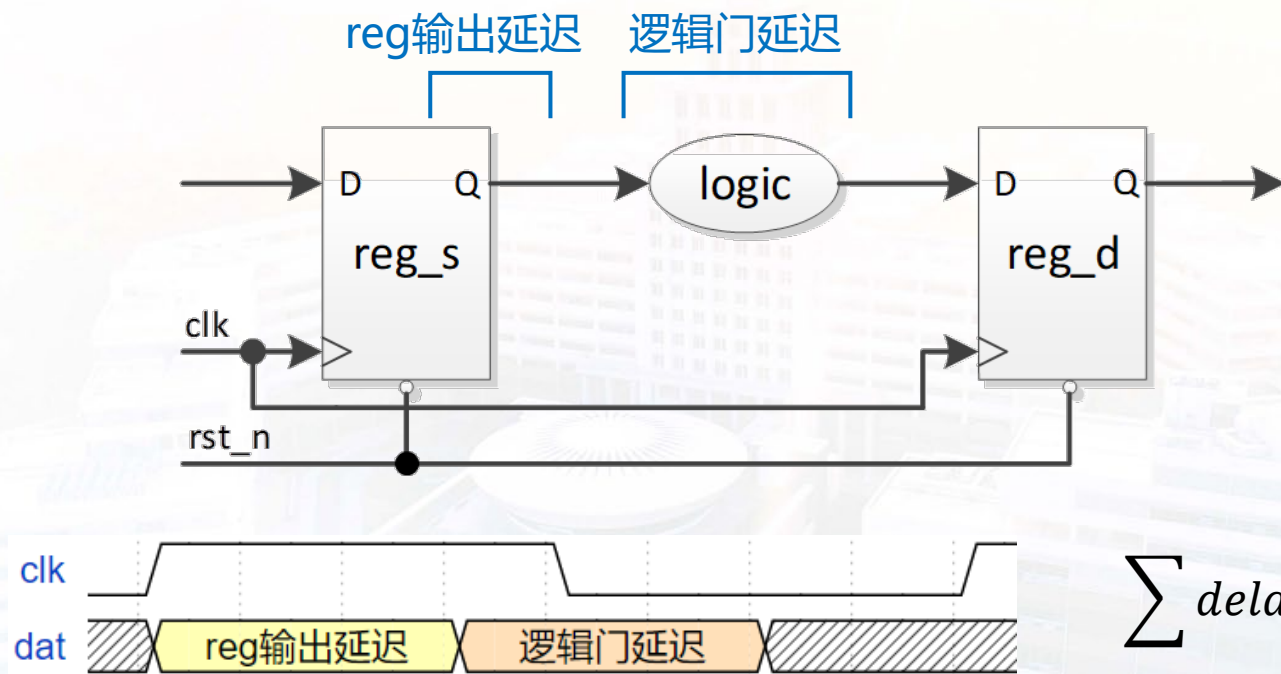


思考

如何提升频率、性能？

引言

□ CPU频率的决定因素

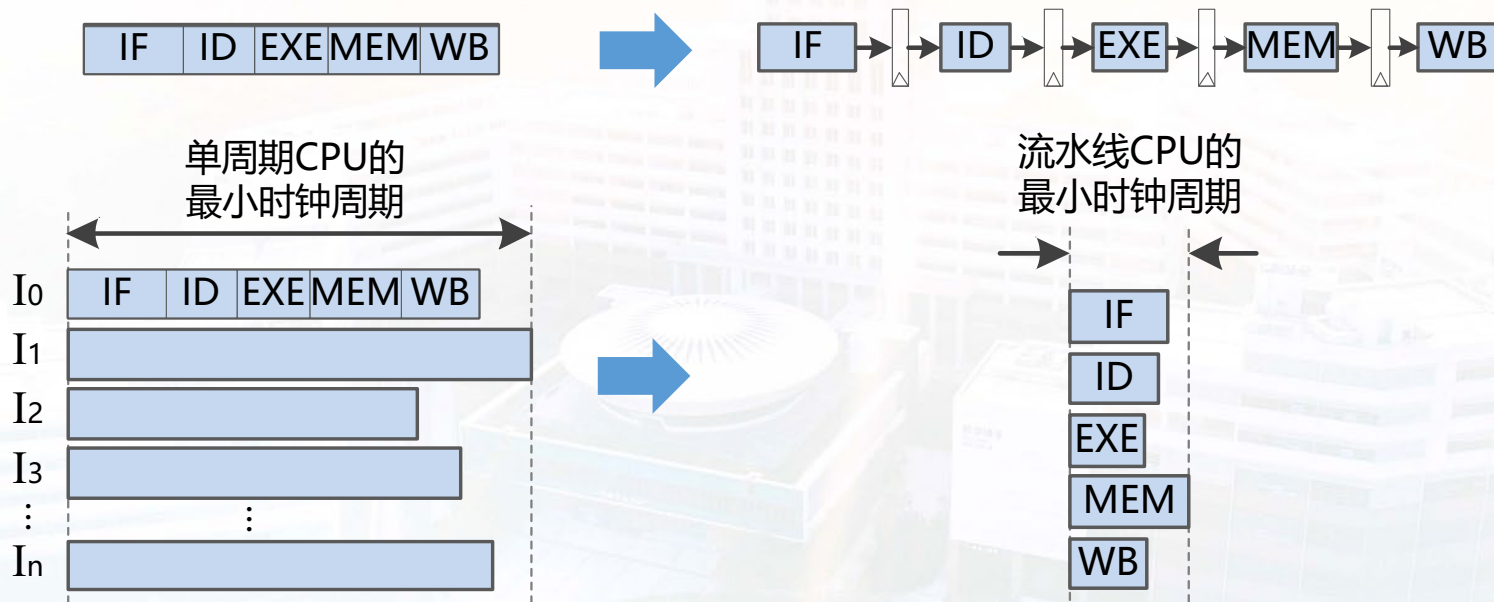


$$\sum delay \leq T_{cycle}$$

引言

□ 流水线CPU

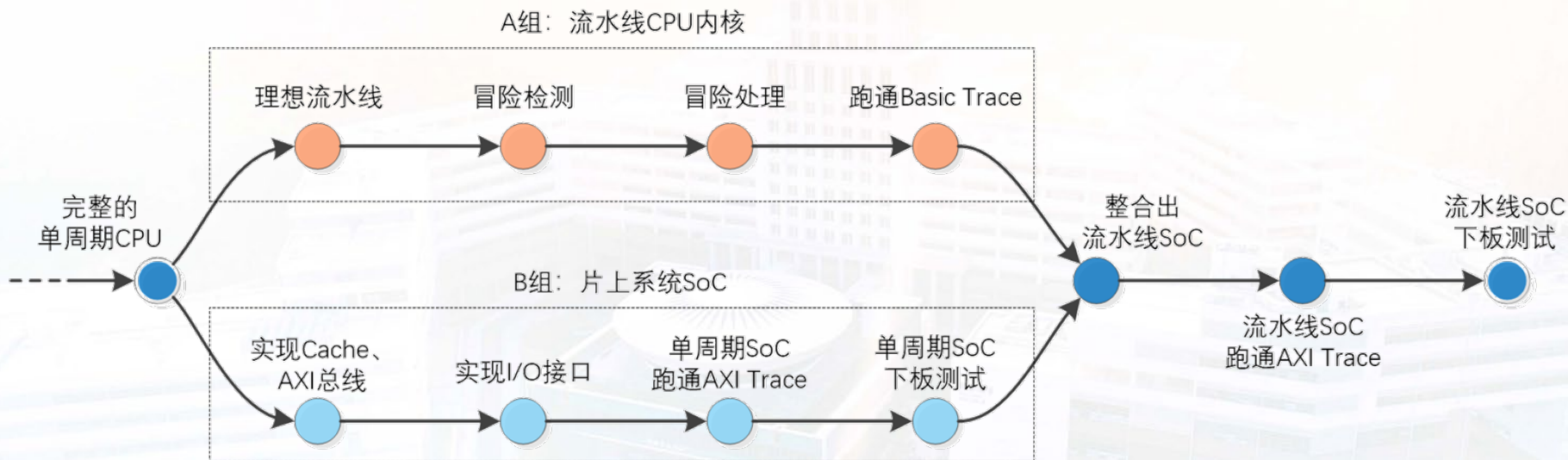
- 将指令处理过程**拆分**为多个阶段，并通过多个阶段**并行执行**来提高性能



引言

□ 设计要求:

A组实现流水线CPU、B组实现SoC，最终实现流水线SoC



引言

□ 设计要求 (A组) :

- 五级流水线 (至少)
- 支持miniRV/miniLA全部指令
- 改造单周期CPU数据通路，实现理想流水线CPU
- 实现静态分支预测，解决控制冒险
- 实现流水线暂停、数据前递机制，解决数据冒险
- 通过调试，使流水线CPU通过Basic Trace测试

引言

□ 设计要求 (B组) :

- 为单周期CPU实现ICache、DCache
- 使用状态机设计实现支持AXI协议的总线控制器
- 为SoC添加总线桥、主存模块
- 完成I/O接口测试程序C_TEST的编写和下板测试
- 实现外设I/O接口电路：拨码开关、LED、数码管、UART、计时器
- 通过调试，使单周期CPU及SoC通过AXI Trace测试

引言

□ 设计要求 (A、B组) :

- 用流水线CPU内核替换掉SoC中的单周期CPU内核
- 通过调试, 使流水线SoC通过AXI Trace测试
- 使用I/O接口程序C_TEST对流水线SoC进行下板测试
- 完成CoreMark或LLAMA2的下板测试
- 尝试优化性能: 提升频率、优化访存、优化计算, etc.

目录



引言

流水线概述

流水线设计

SoC架构及系统总线

I/O接口电路

流水线概述

□ 流水线：

- 将指令处理过程**拆分**为多个步骤，并通过多个硬件处理单元**并行执行**来加快指令执行速度

□ 理想条件下，流水线CPU的指令执行时间：

- $T_{\text{non-pipeline}}$ ：非流水线指令执行时间
- T_{Pipeline} ：流水线指令执行时间
- N ：流水线级数

$$T_{\text{Pipeline}} = \frac{T_{\text{non-pipeline}}}{N}$$

流水线概述 – 流水线划分

□ 流水线划分原则

1. 按**功能**划分

- 各段有明确的功能，或存在逻辑上具备先后关系的若干个阶段

2. 按**延时**划分

- 为提高频率/性能，各段延时应尽量接近

3. 按**位宽**划分

- 松耦合原则：各段之间的交互信号应尽量少

流水线概述 – 流水线划分案例

□ RISC经典五级流水 (MIPS、RISC-V、ARM8/9)

IF -> ID -> EXE -> MEM -> WB

- ① **取指**：从指令存储器读取一条指令
- ② **译码**：分解指令、从寄存器堆取出操作数、产生控制信号
- ③ **执行**：根据控制信号完成相应的运算、计算出访存地址
- ④ **访存**：访问数据存储器
- ⑤ **写回**：将执行阶段的结果或访存取出的数据写回寄存器堆

流水线概述 – 流水线划分案例

□ ARM10六级流水

IF -> **ISS** -> ID -> EXE -> MEM -> WB

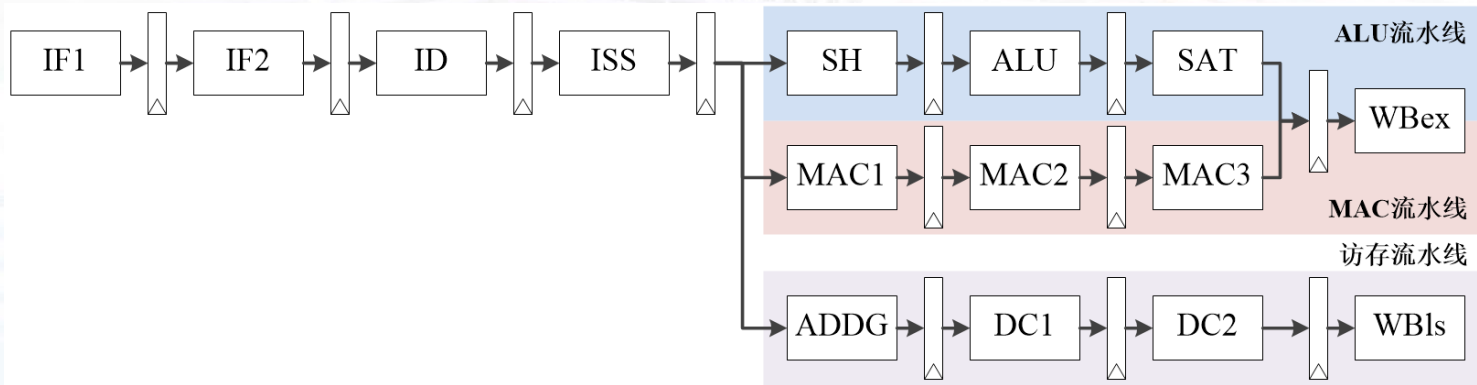
- ① **取指**: **分支预测**、从指令存储器读取一条指令
- ② **分派**: 判断指令是否是协处理器指令
- ③ **译码**: 分解指令、从寄存器堆取出操作数、产生控制信号
- ④ **执行**: 根据控制信号完成相应的运算、计算出访存地址
- ⑤ **访存**: 访问数据存储器
- ⑥ **写回**: 将执行阶段的结果或访存取出的数据写回寄存器堆

流水线概述 – 流水线划分案例

□ ARM11八级流水

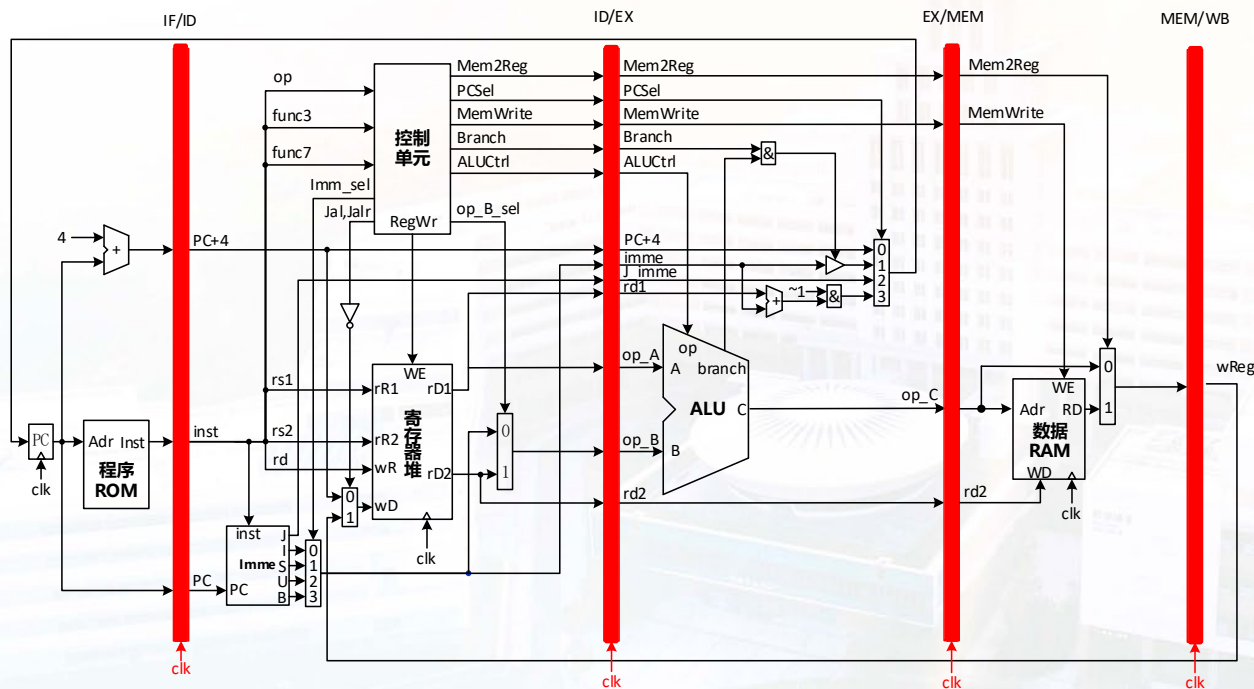
IF1 -> IF2 -> ID -> ISS -> ALU流水线/MAC流水线/访存流水线

- ① **取指1**：动态分支预测、从指令存储器读取一条指令
- ② **取指2**：静态分支预测
- ③ **译码**：分解指令、判断指令是否是协处理器指令
- ④ **分派**：从寄存器堆取出操作数，并分派指令到子流水线



流水线概述 – 流水线寄存器

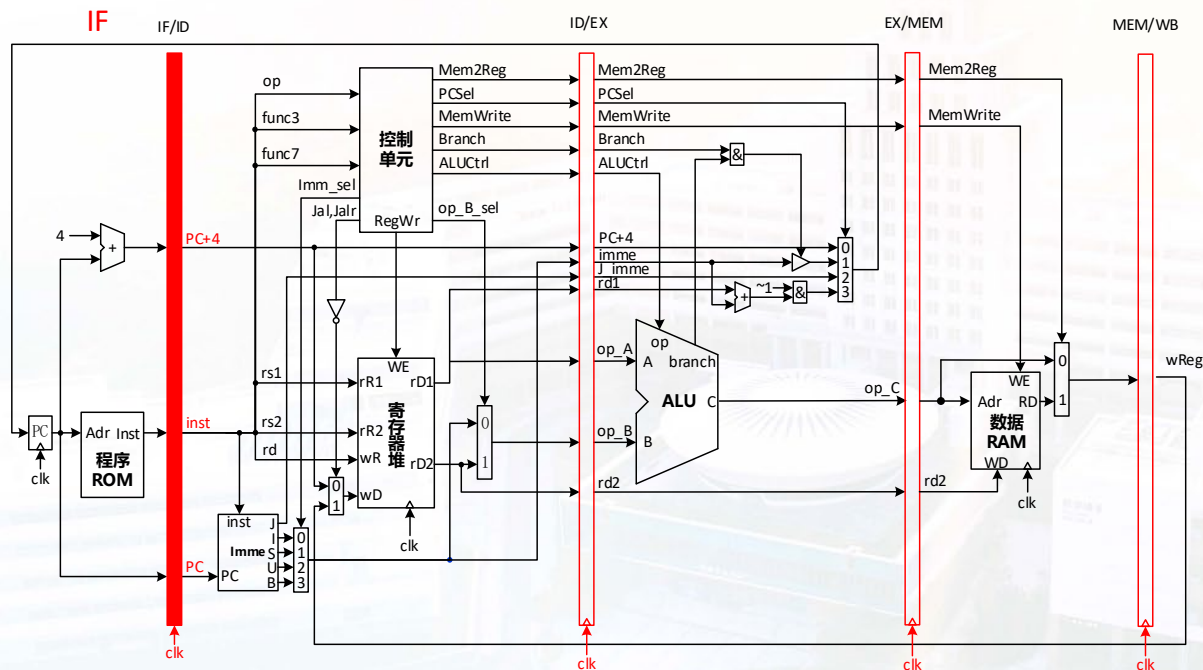
□ 流水寄存器 / 段寄存器



- 切割组合逻辑，提升系统频率
- 保存该流水级输出数据信息，交给下一级处理，并共享给其他指令

流水线概述 – 流水线寄存器

□ 举例：IF/ID流水寄存器



- 段寄存器暂存什么信号，需根据具体设计而定
- 可以由多个“小”寄存器“拼”成完整的段寄存器

流水线概述 – 流水线寄存器

□ IF/ID流水线寄存器

IF/ID



PC: J型指令、B型指令

PC+4: J型指令

inst: 所有指令

参考RTL实现

```
1 always @ (posedge clk or posedge rst) begin
2     if (rst) id_pc <= 32'h0;
3     else     id_pc <= if_pc;
4 end
5
6 always @ (posedge clk or posedge rst) begin
7     if (rst) id_inst <= 32'h0;
8     else     id_inst <= if_inst;
9 end
```

目录



引言

流水线概述

流水线设计

SoC架构及系统总线

I/O接口电路

理想流水线

流水线冒险

静态分支预测

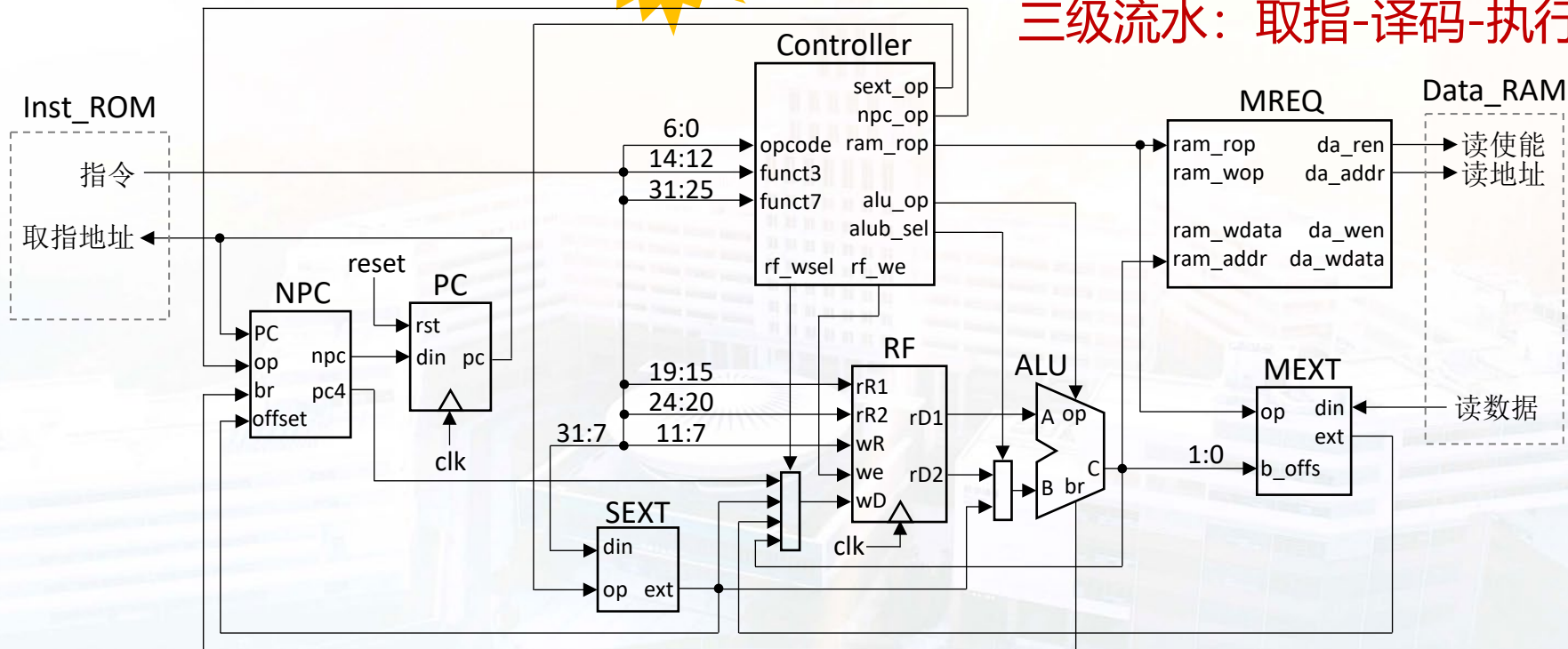
流水线设计 – 单周期改造

举个栗子



□ 单周期 → 理想流水线 How?

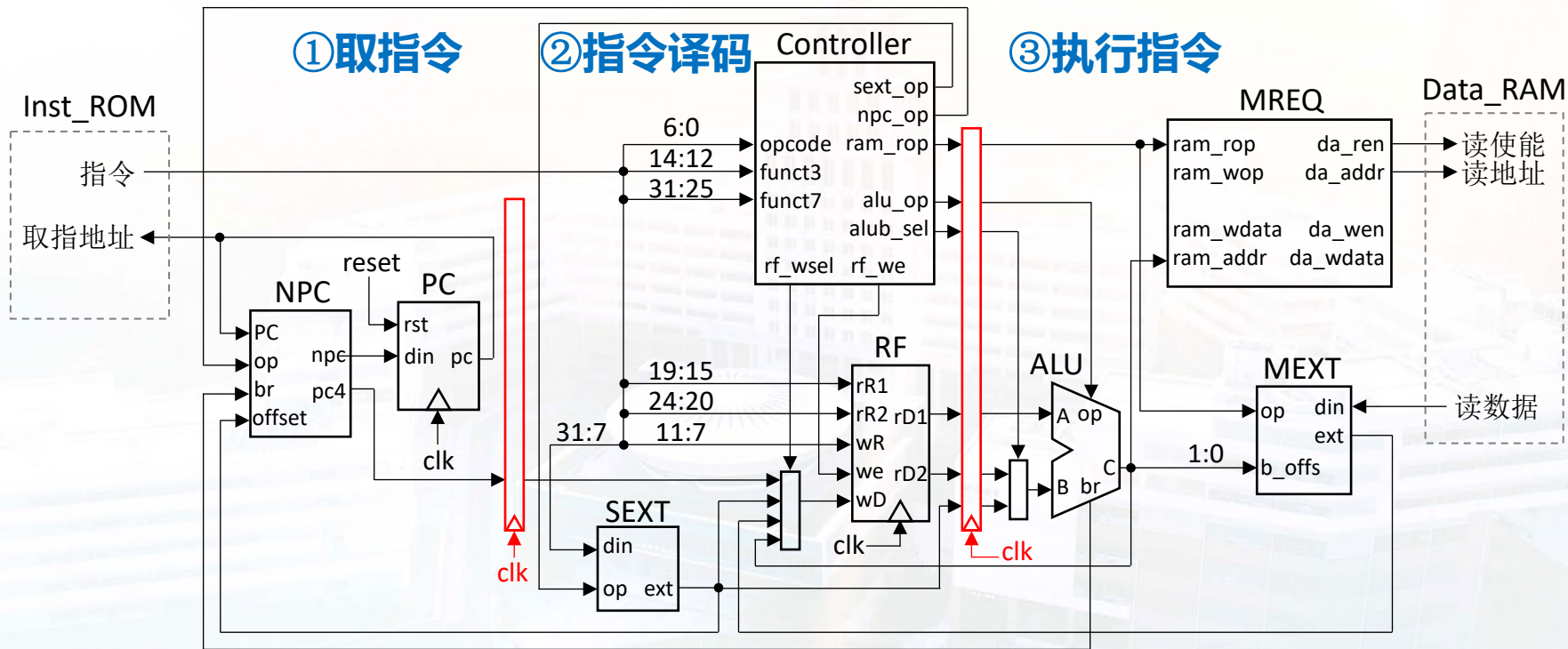
三级流水：取指-译码-执行



流水线设计 – 单周期改造

单周期 → 理想流水线

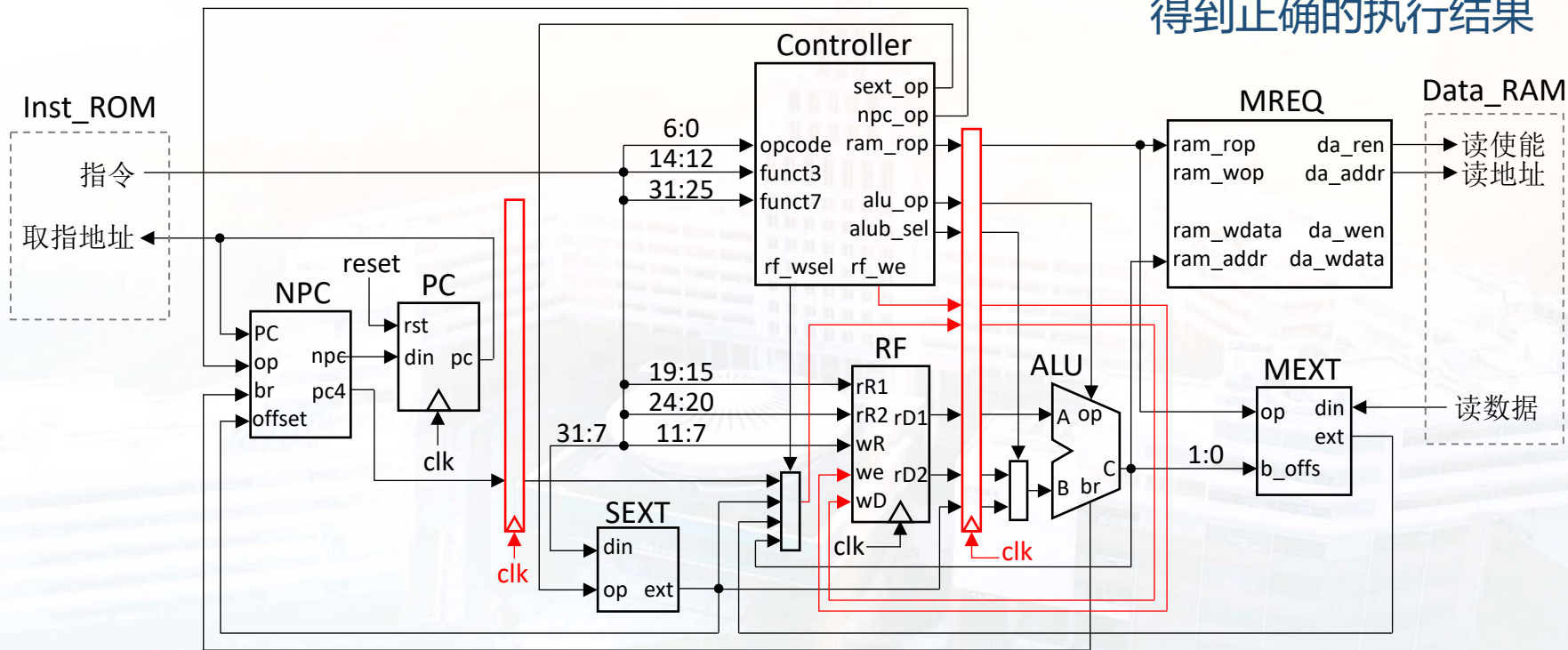
- Step1: 切分数据通路，添加段寄存器 (红色为新增)



流水线设计 – 单周期改造

单周期 → 理想流水线

- **Step2: 写回逻辑延后至最后阶段 (红色为新增)** ➤ 延后写回逻辑，才能得到正确的执行结果



目录



引言

流水线概述

流水线设计

SoC架构及系统总线

I/O接口电路

理想流水线

流水线冒险

静态分支预测

流水线设计 – 冒险

□ 相关 (Dependence)：指令之间存在依赖关系

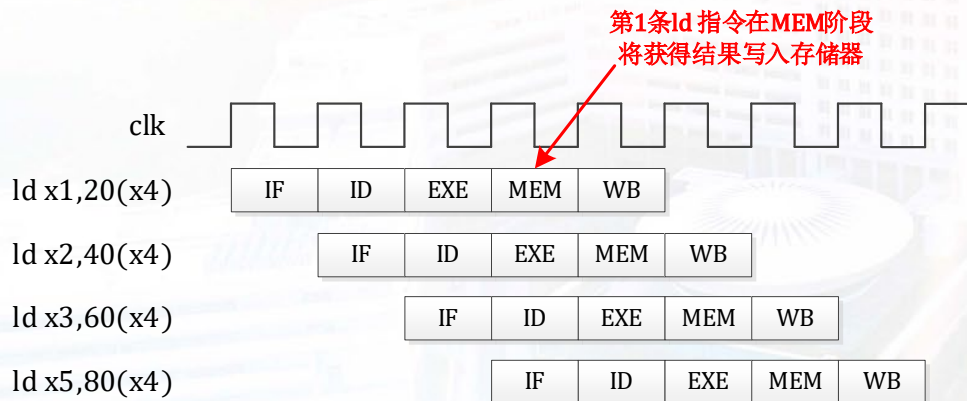
- ① 数据相关 —— 当前指令使用前面指令的执行结果
- ② 名相关
 - 多条指令使用相同的寄存器/存储单元，但不存在数据相关
 - 包括反相关（先读后写）、输出相关
- ③ 控制相关 —— 分支指令

□ 冒险 (Hazard)：指令相关引起流水线工作异常的现象

流水线设计 – 冒险

□ 结构冒险

- 因**缺乏硬件支持**而导致指令不能在预定的时钟周期内执行的现象
- 比如：在冯·诺依曼架构中，流水线中只有一个存储器，数据与指令共享



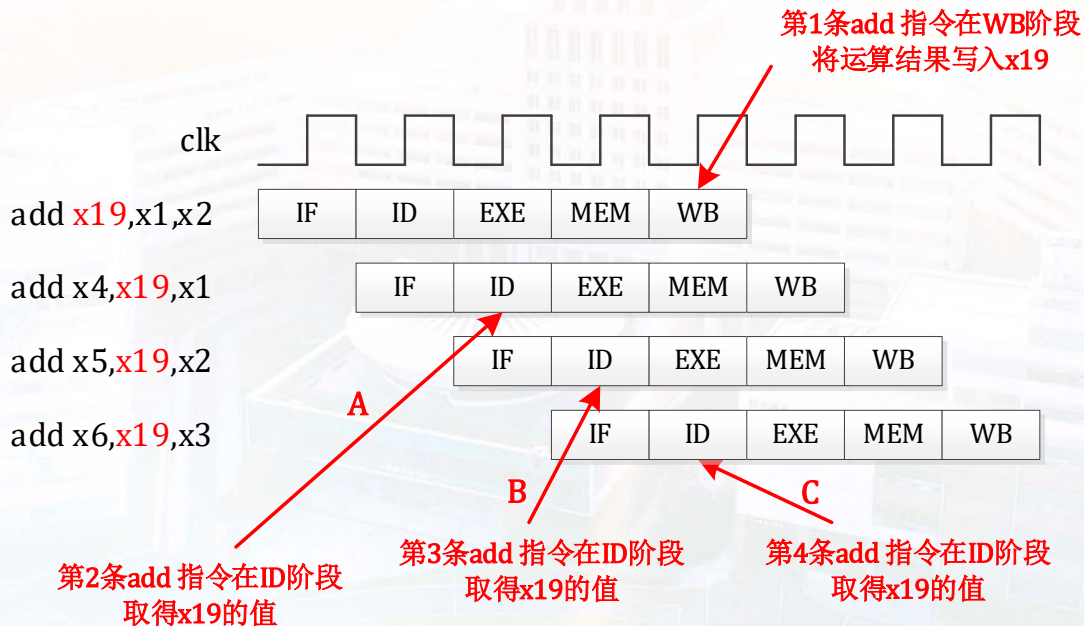
解决方法：

冯·诺依曼结构 -> 哈佛结构

流水线设计 – 冒险

□ 数据冒险

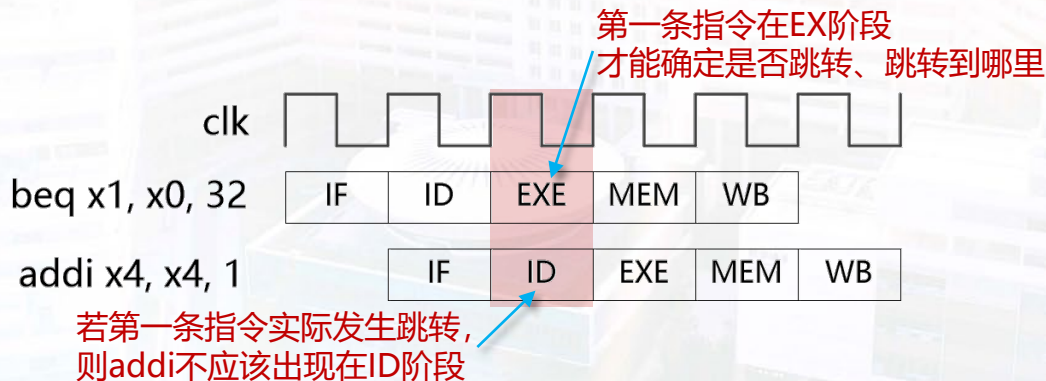
- 一条指令依赖于前面一条尚在流水线中的指令，因**指令无法获取执行所需数据**，导致指令不能在预期的时钟周期内执行的现象



流水线设计 – 冒险

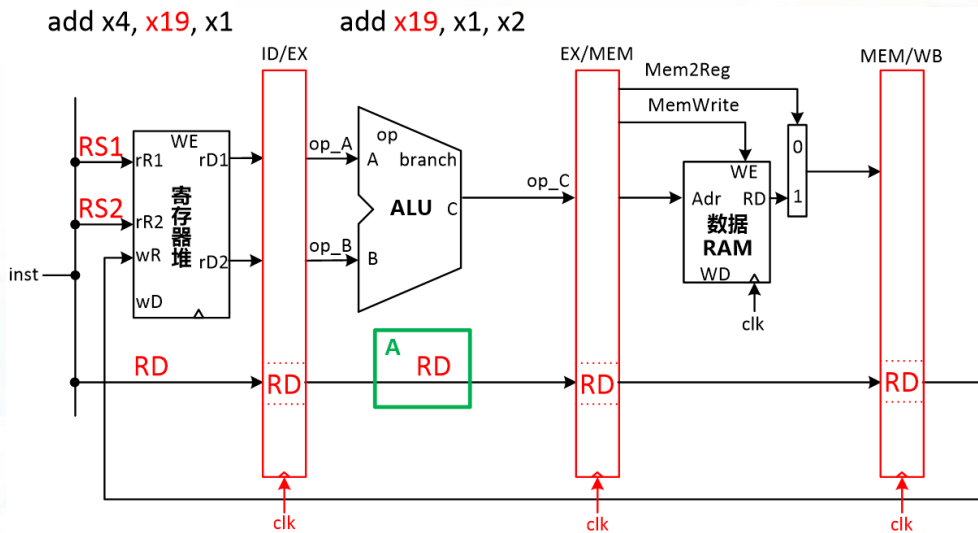
□ 控制冒险

- 由于取到的指令并不是所需要的，或者指令地址的流向不是流水线所预期的，导致正确的指令无法在正确的时钟周期内执行的情况
- 分支指令后的IF段依赖于分支的结果，无法保证永远取到正确的指令

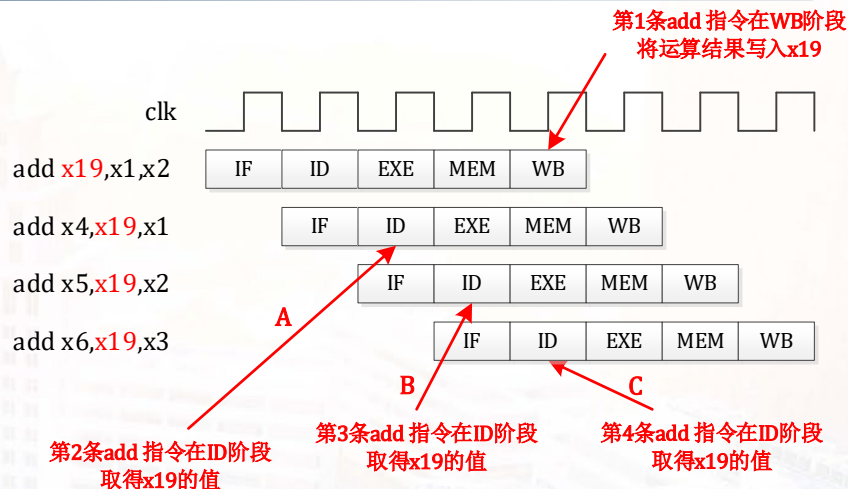


流水线设计 – 数据冒险检测

□ RAW冒险检测 – 情形A



注意: $REG_{ID/EX}.RD$ 表示 ID/EX 寄存器的 RD 字段
 $ID.RS_x$ 表示 ID 阶段 RS_x 寄存器的寄存器号



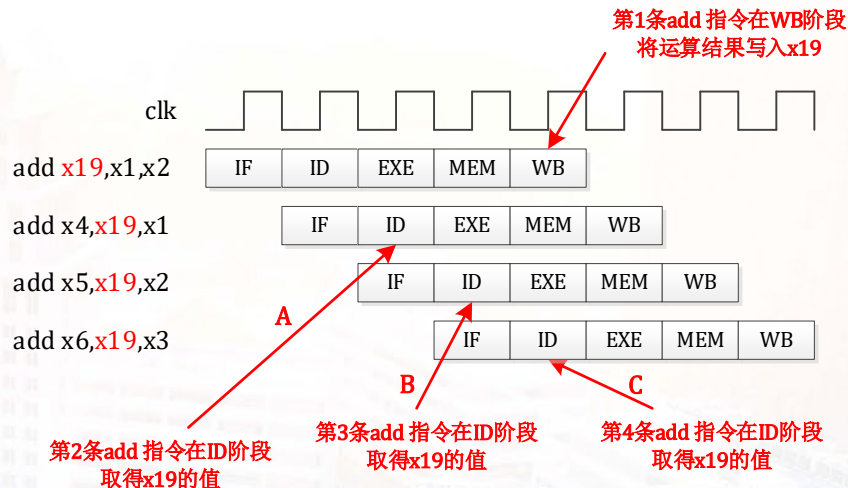
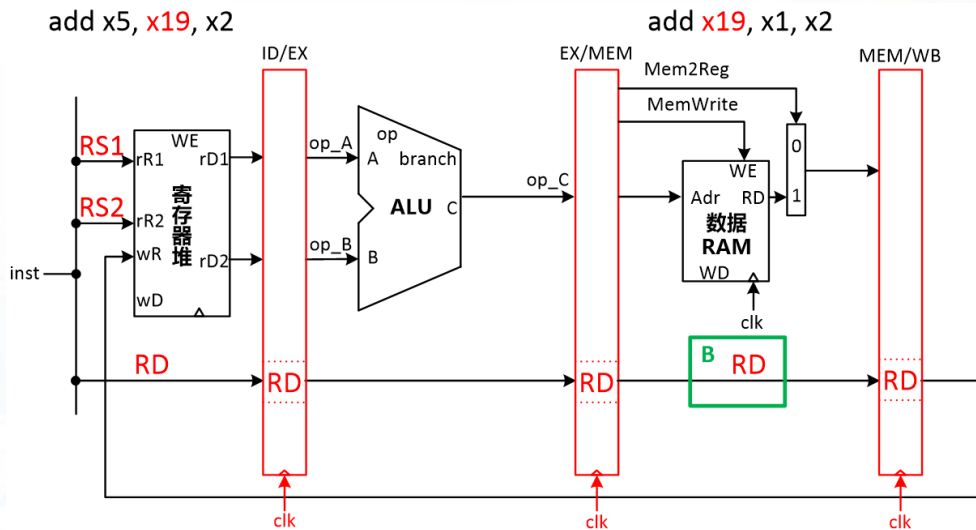
判断条件:

$$REG_{ID/EX}.RD == ID.RS1$$

或 $REG_{ID/EX}.RD == ID.RS2$

流水线设计 – 数据冒险检测

□ RAW冒险检测 – 情形B



判断条件:

$$REG_{EX/MEM}.RD == ID.RS1$$

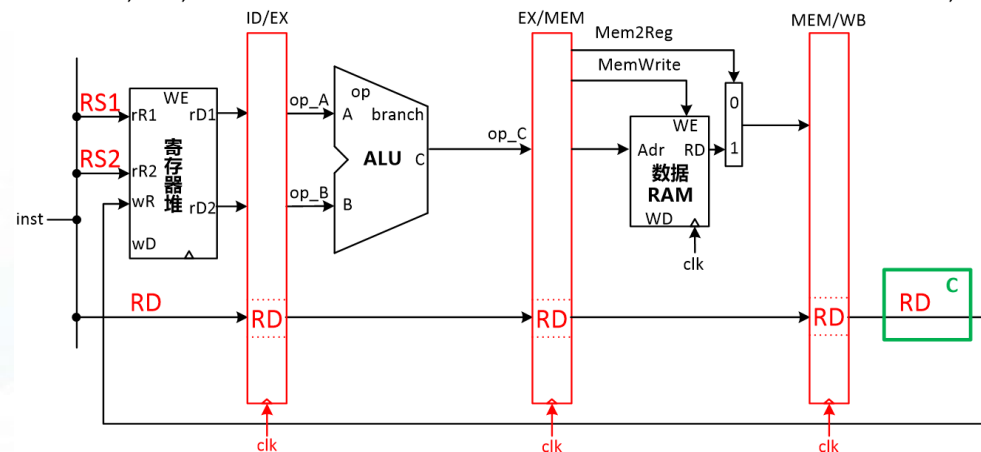
$$\text{或 } REG_{EX/MEM}.RD == ID.RS2$$

流水线设计 – 数据冒险检测

□ RAW冒险检测 – 情形C

add x6, x19, x3

add x19, x1, x2



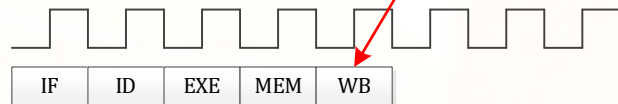
clk

add x19, x1, x2

add x4, x19, x1

add x5, x19, x2

add x6, x19, x3



第2条add指令在ID阶段取得x19的值

第3条add指令在ID阶段取得x19的值

第4条add指令在ID阶段取得x19的值

判断条件:

$$REG_{MEM/WB}.RD == ID.RS1$$

$$\text{或 } REG_{MEM/WB}.RD == ID.RS2$$

流水线设计 – 数据冒险检测

□ RAW冒险检测 – 情形C

判断条件:

$$REG_{MEM/WB}.RD == ID.RS1$$

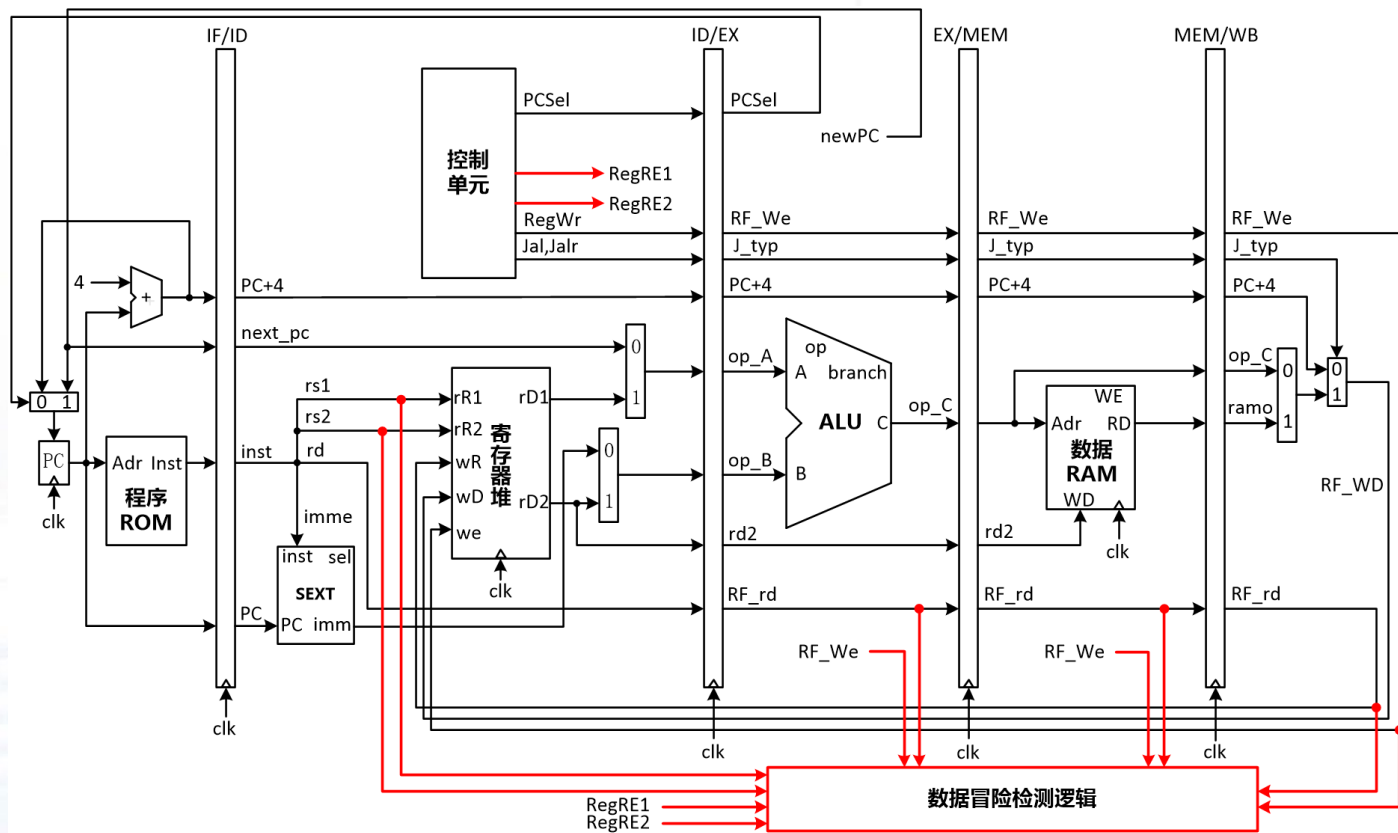
或 $REG_{MEM/WB}.RD == ID.RS2$

RTL参考实现

```
1 wire rs1_id_wb_hazard = (wb_rd == id_rs1) & wb_we & id_rf1 & (wb_rd != 5'h0);  
2 wire rs2_id_wb_hazard = (wb_rd == id_rs2) & wb_we & id_rf2 & (wb_rd != 5'h0);
```

- ◆ U型、J型指令**没有源操作数寄存器**，不需读取RF，故不存在RAW
- ◆ 单周期的RF采用**异步读取**，可能造成**误判**
- ◆ 为了保证冒险检测的正确性，需添加“读”标志信号加以区分
- ◆ 零号寄存器不可写，故其不存在RAW冒险

流水线设计 – 数据冒险检测逻辑



仅为示例，
请在理解的
基础上完成
自己的设计

流水线设计 – 数据冒险解决

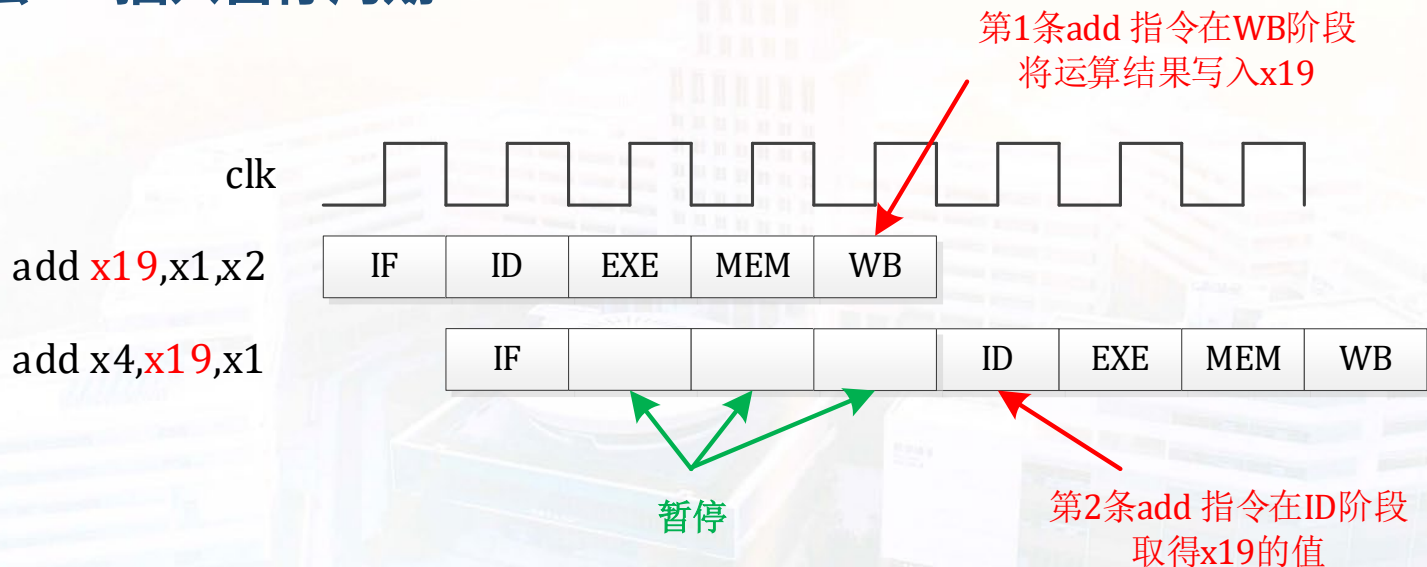
□ 数据冒险解决方法

- ① 停顿 / 暂停
- ② 数据前递 / 前推 / 旁路 / 转发
- ③ 乱序执行

流水线设计 – 暂停法

□ 解决数据冒险

方法1：插入暂停周期



流水线设计 – 暂停法

□ 解决数据冒险

方法1：插入暂停周期

实现方法：

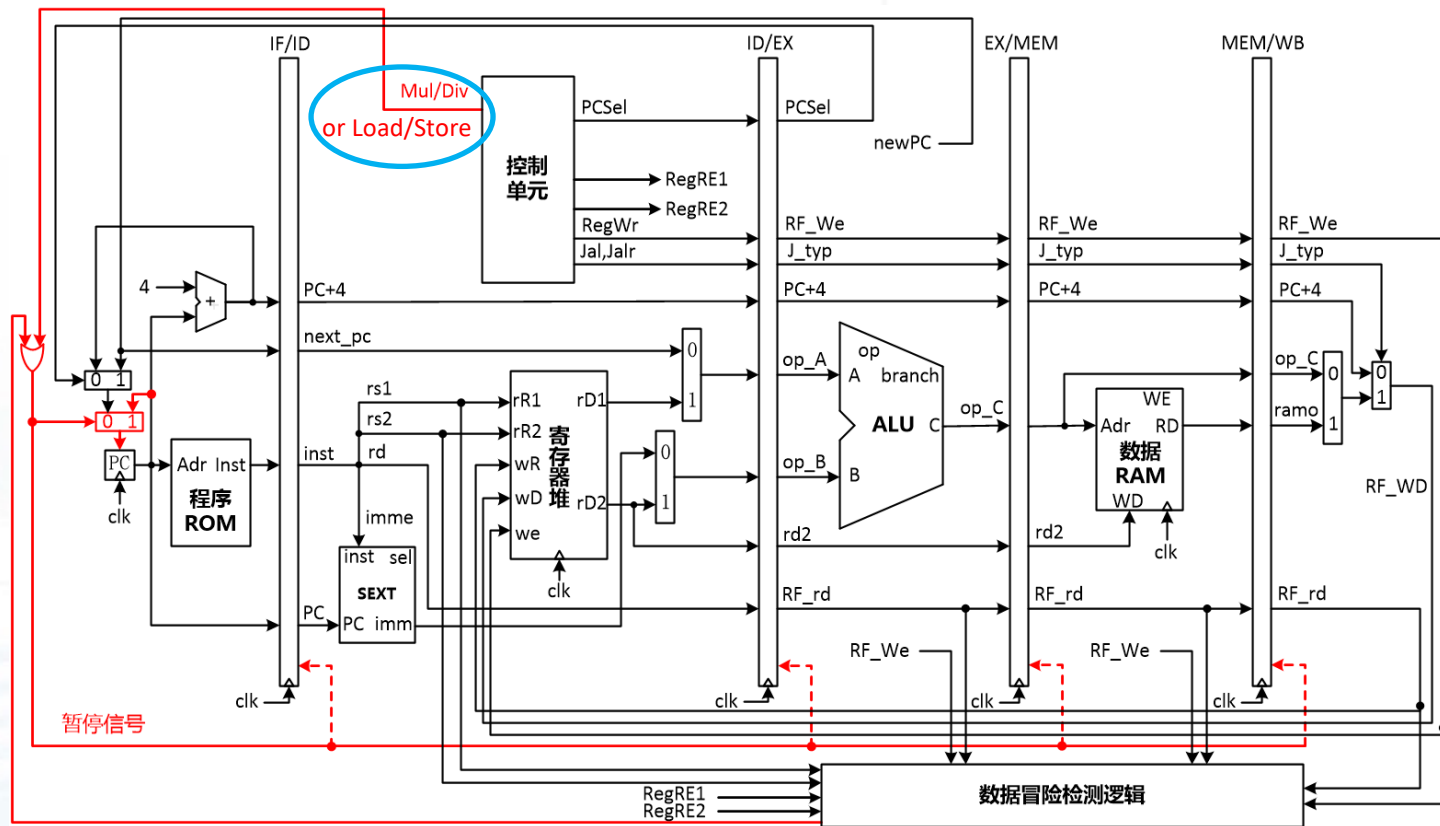
保持PC的值不变

保持流水线的各个段寄存器不变

- IF/ID、ID/EX、EX/MEM、MEM/WB

暂停寄存器：将寄存器的输入修改为寄存器自身的输出

流水线设计 – 暂停法设计



仅为示例，
请在理解的
基础上完成
自己的设计

流水线设计 – 暂停法设计

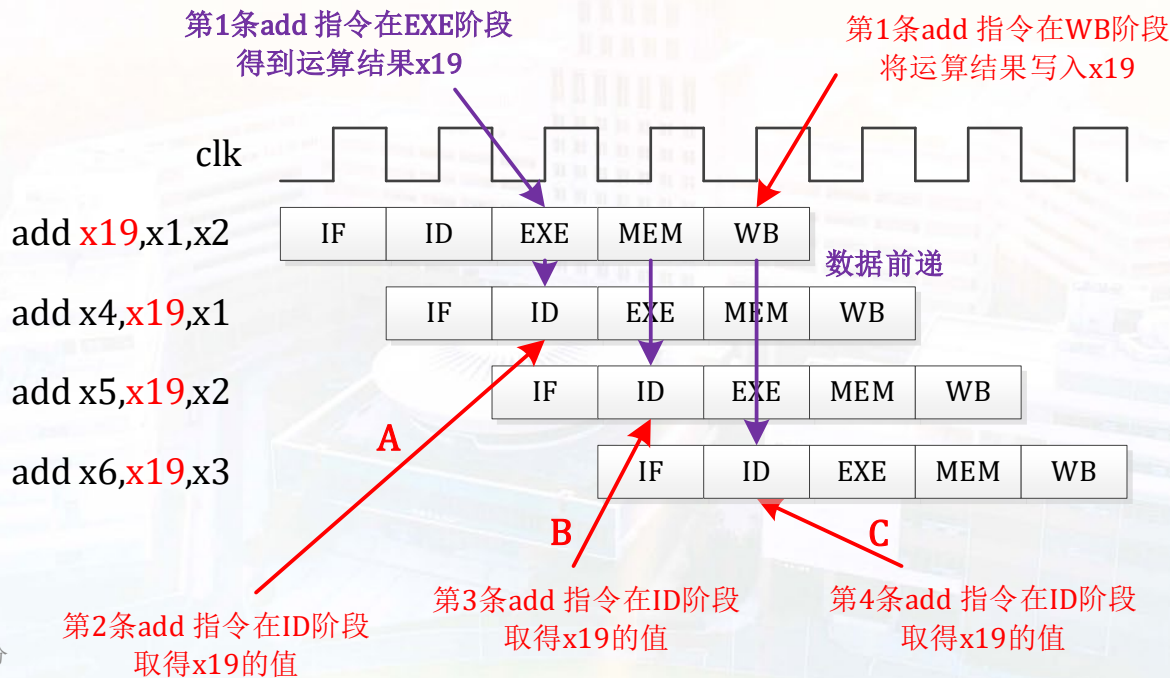
□ 暂停法除了解决数据冒险，还可用于实现**访存指令**、**乘除法指令**

- 访存指令需等待总线和存储器完成读写操作，乘除法指令需等待乘除法器完成运算，都需要许多个时钟周期才能完成
- 实现思路：MEM阶段检测到访存指令（或EX阶段检测到乘除法指令）后，**暂停前面的流水线阶段**，直到访存（或乘除运算）完毕

流水线设计 – 数据前递法

□ 解决数据冒险

方法2：数据前递

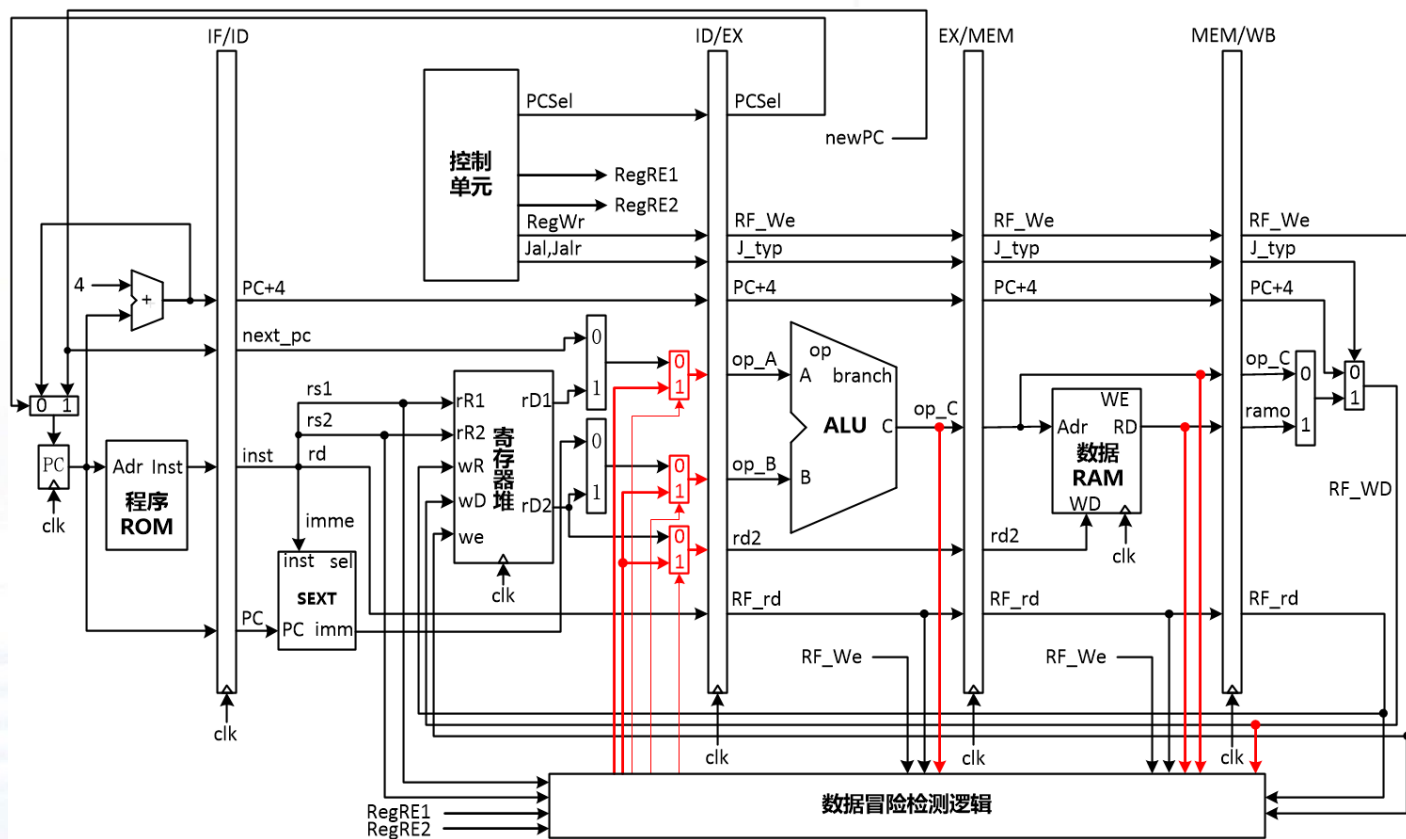


流水线设计 – 数据前递法

□ 数据前推的逻辑

- 将EXE、MEM、WB阶段的结果前推到ID阶段，参与ID阶段运算源操作数的选择
- **Pros:** 重叠执行程度高、发生冒险时性能损失小
- **Cons:** 逻辑复杂，影响频率的提升

流水线设计 – 数据前递法



仅为示例，
请在理解的
基础上完成
自己的设计

流水线设计 – 乱序法

□ 解决数据冒险

方法3：乱序执行法

存在数据冒险

```
add x2, x1, x6  
add x3, x2, x7  
add x4, x1, x8  
add x5, x9, x7  
add x6, x7, x1
```



无数据冒险

```
add x2, x1, x6  
add x4, x1, x8  
add x5, x9, x7  
add x6, x7, x1  
add x3, x2, x7
```

无相关指令

流水线设计 – 乱序法

□ 解决数据冒险

方法3：乱序执行法

实现方法：

- ① 软件调度 (汇编器、编译器)
- ② 硬件前瞻执行 (分支预测、Tomasulo算法)

参考资料：《计算机系统结构教程(第2版)》张晨曦

目录



引言

流水线概述

流水线设计

SoC架构及系统总线

I/O接口电路

理想流水线

流水线冒险

静态分支预测

流水线设计 – 静态分支预测

□ 控制冒险

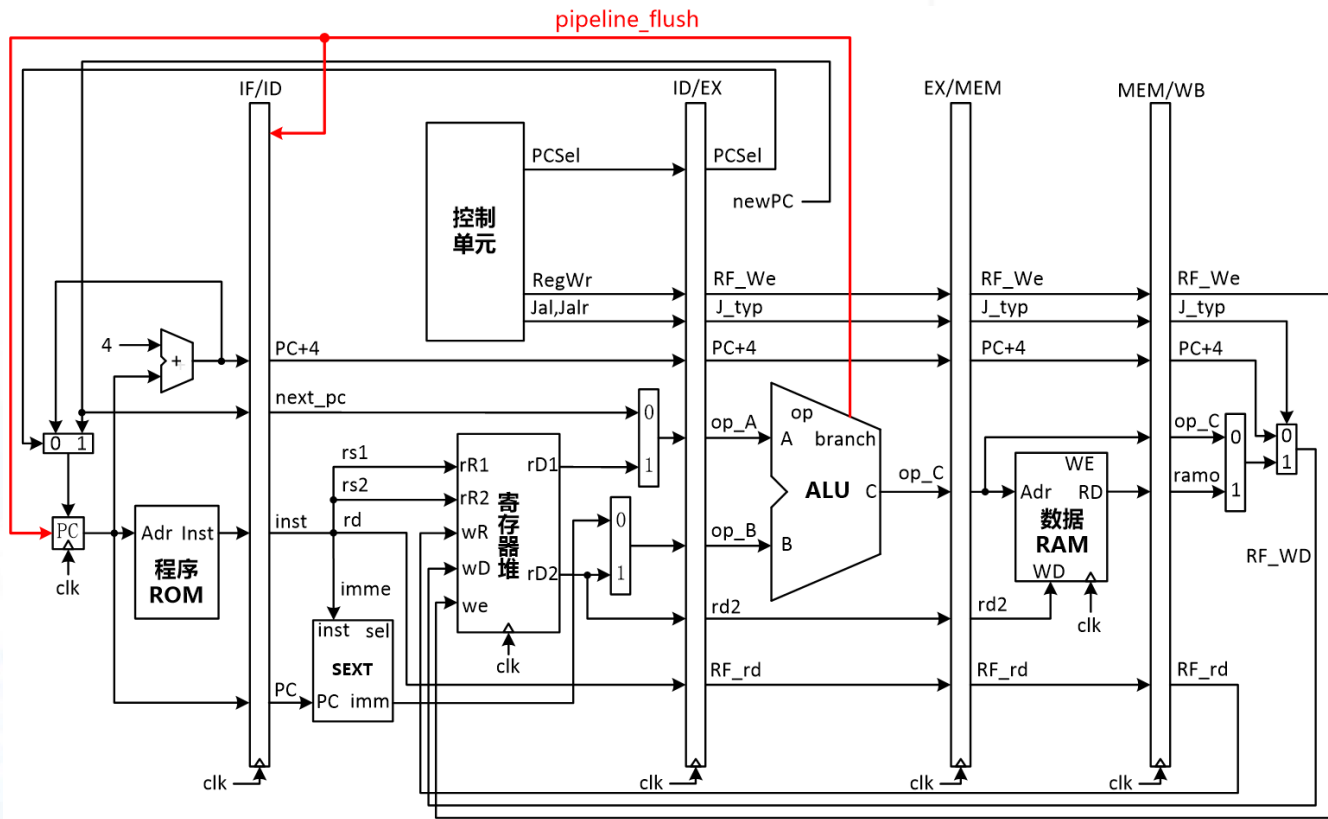
- 分支指令执行结果出来之前，无法确定接下来要取哪条指令
- 解决思路：
 - ① 暂停法 简单，但损失性能
 - ② 延迟槽 降低代码可读性、性能提升有限 **RISC-V已舍弃不用**
 - ③ 分支预测

流水线设计 – 静态分支预测

□ 解决控制冒险

- RISC-V和LoongArch默认采用预测指令总是不跳转的静态预测
- 默认以PC+4不断顺序取指执行
- 发现指令要发生跳转时，清除那些不应该执行的指令

流水线设计 - 静态分支预测



预测失败时，需排空IF、ID

仅为示例，请在理解的
基础上完成自己的设计

目录



引言

流水线概述

流水线设计

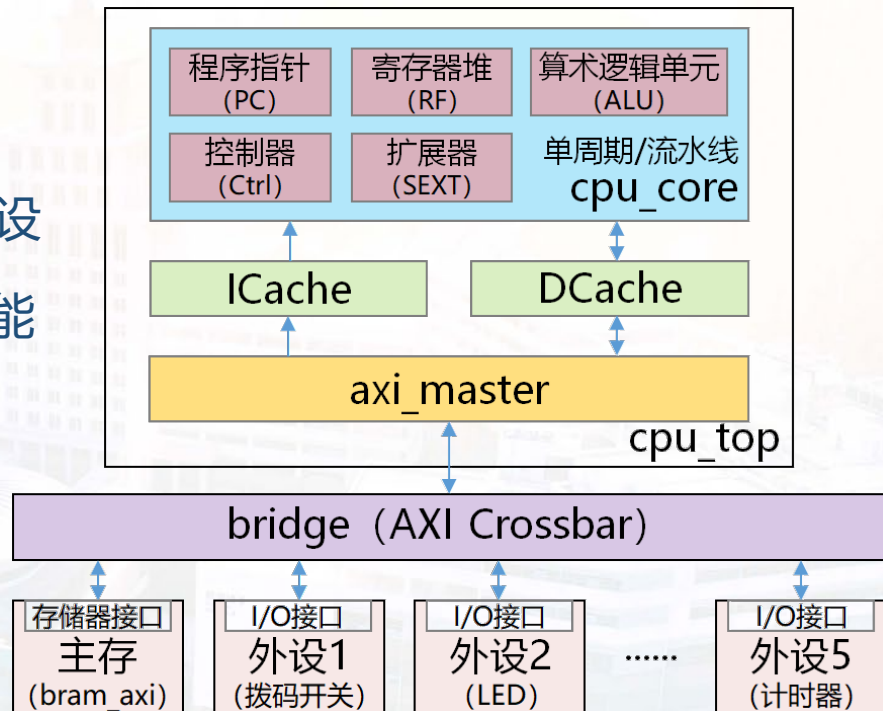
SoC架构及系统总线

I/O接口电路

SoC架构及系统总线 – SoC架构

□ 本课程将实现的SoC架构:

- CPU的主要功能是控制和运算
- CPU通过总线才能访问主存、外设
- Cache缓解存储墙，提高访存性能
- 总线桥实现一对多连接
- I/O接口实现CPU与外设的交互



SoC架构及系统总线 – 系统总线

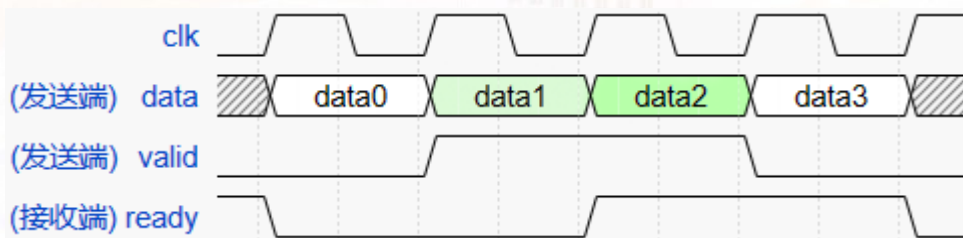
□ 系统总线是CPU和主存实现数据交互的通路

- 系统总线的性能直接影响CPU的访存性能
- 本课程采用AXI总线作为系统总线
 - AXI是基于ready-valid握手协议的高性能嵌入式总线
 - 读、写通道独立，支持并行读写
 - 支持猝发传输、乱序访问

SoC架构及系统总线 – 系统总线

□ ready-valid握手协议

- ready信号：接收端的**就绪**状态
- valid信号：发送端发送数据的**有效**状态

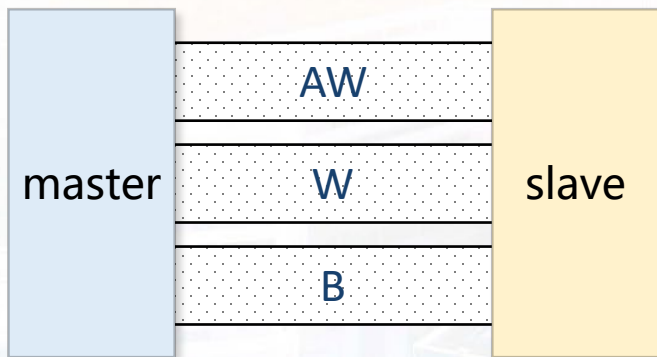


- 对发送端：当且仅当valid有效时，数据才有效 —— data1、data2是有效数据
- 对接收端：当且仅当ready有效时，才可能接收数据 —— 只有data2被接收
- ◆ 当且仅当**valid、ready同时有效**时，数据才能在clk上升沿时成功传输

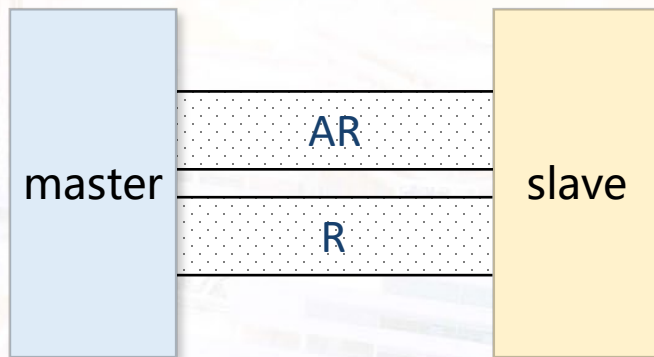
SoC架构及系统总线 – 系统总线

□ AXI4总线协议

- 包含5个独立通道：写地址(**AW**)、写数据(**W**)、写响应(**B**)、读地址(**AR**)、读数据(**R**)



3个通道用于写请求



2个通道用于读请求

- 每个通道都基于ready-valid信号实现握手

目录



引言

流水线概述

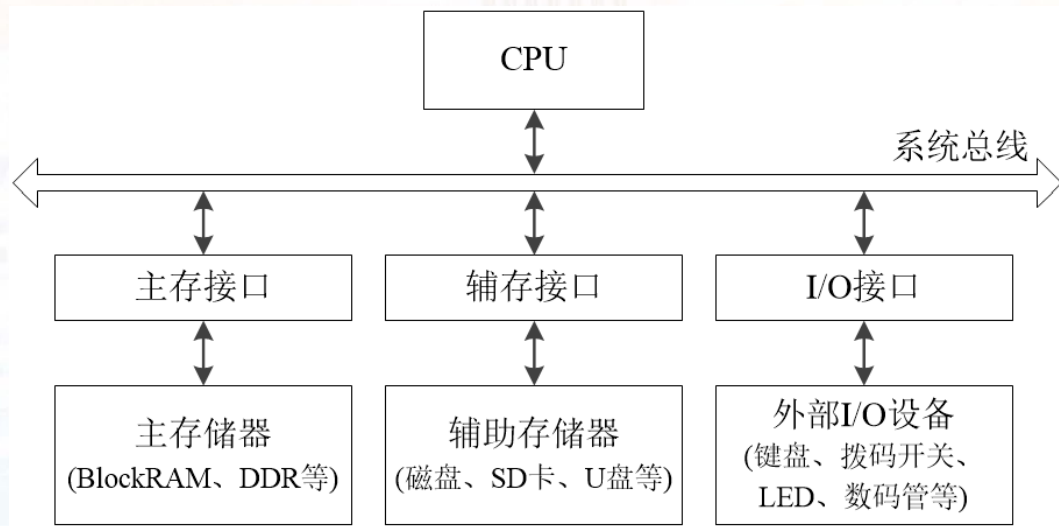
流水线设计

SoC架构及系统总线

I/O接口电路

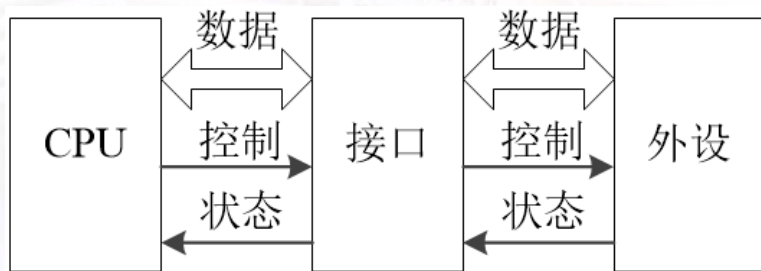
SoC设计 – I/O接口

- ◆ CPU通过接口连接“外部世界”，接口负责在CPU与“外部世界”之间中转各种信息
 - ◆ 接口包括存储器接口、I/O接口



SoC设计 – I/O接口

- ◆ CPU通过接口连接“外部世界”，接口负责在CPU与“外部世界”之间中转各种信息
 - ◆ 接口受CPU控制，外部设备受接口控制
 - ◆ I/O接口电路“抹平”外设与CPU之间的数据及时序差异

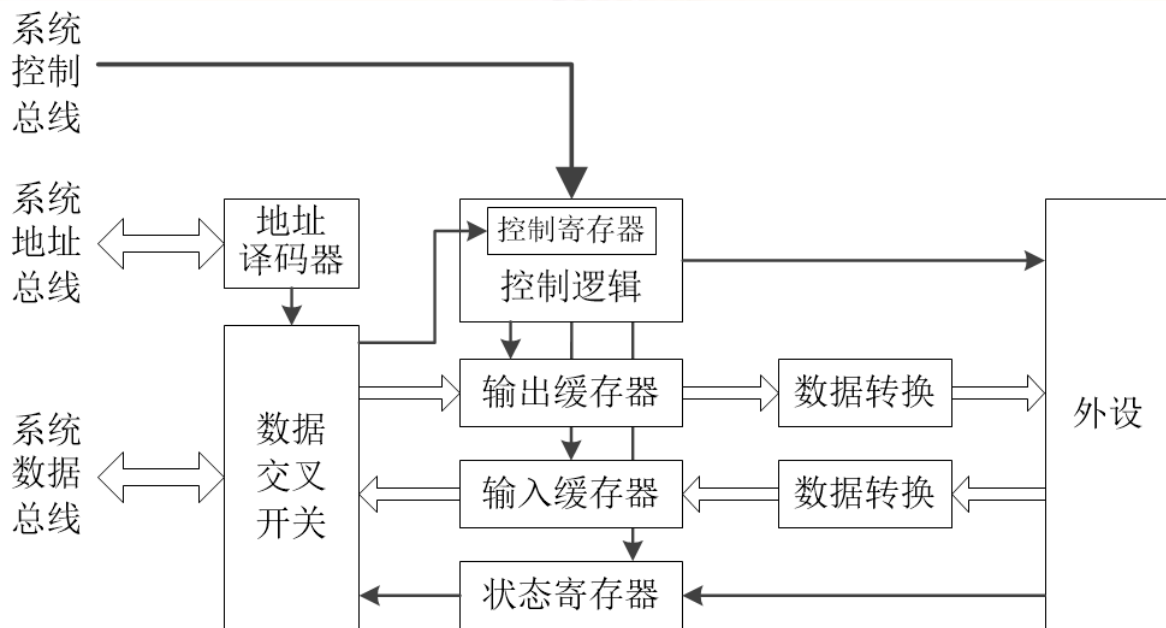


SoC设计 – I/O接口的功能

- ◆ 信号转换功能/预处理功能
 - ◆ 完成总线信号与I/O设备信号之间的转换，如电平转换、串并转换等
- ◆ 数据缓存功能
 - ◆ 缓存CPU和外设之间的数据，相应的缓存器称为数据口
- ◆ 接受和执行CPU命令的功能
 - ◆ 使用寄存器存放来自CPU的命令，该寄存器称为命令口
- ◆ 控制和监视外设执行的功能
 - ◆ 状态寄存器存储外设状态，称为状态口
- ◆ 设备选择功能/选址功能
 - ◆ 根据访问地址选择相应的I/O接口或接口中的设备

SoC设计 – I/O接口的结构

- ◆ I/O接口电路的核心：数据口、命令口、状态口、控制逻辑
- ◆ CPU通过地址访问外设——**基地址确定I/O接口，偏移地址确定具体端口**





HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HITSZ